Diabetes Readmission Intervention Queue

Build an enterprise backend that converts historical healthcare data into an actionable care-management workflow.

Recommended effort	Core stack	Role
4-6 hours	**Python • FastAPI • PostgreSQL**	**Forward Deployed Engineer**

The FDE Challenge

Do not build a model first. Build the enterprise workflow around the problem: ingest data, apply an explainable prioritization rule, persist it, expose it through APIs, and measure whether the prioritization aligns with historical outcomes.

This is a learning prototype. It is not a clinically validated scoring system and must not be used for real patient care decisions.

1. Enterprise Scenario

A hospital has historical diabetes encounter data and is concerned about patients returning within 30 days of discharge. The care-management team cannot manually review every discharged patient.

Your role is to act as the FDE assigned to create the first operational MVP. The application should help the hospital answer:

Primary Business Decision

Which recently discharged diabetes patients should the care-management team review first?

2. Users and Business Outcome

User	What They Need	Business Outcome
Care Management Team	Prioritized review queue	Focus limited capacity on higher-risk cases
Hospital Operations	Queue and workload visibility	Understand intervention volume and backlog
Clinical Leadership	Explainable rationale	Review whether prioritization is operationally sensible
Finance / Transformation	Outcome and value signals	Assess whether the intervention is worth scaling

3. FDE Discovery Before Coding

Before you write code, add a short section to your README covering the following three areas.

What the dataset tells us	What we do NOT know	MVP hypothesis
Historical encounters	Which readmissions were preventable?	Prior utilization may help prioritize follow-up
Historical <30-day readmissions	Cost of a readmission?	Care-management capacity is limited
Prior healthcare utilization	Current discharge workflow?	A risk queue may be better than reviewing everyone
Operational / treatment attributes	Cost of intervention?	Rules are acceptable for an explainable MVP
Historical outcome	Clinical validity of rules?	Historical outcomes can be used to evaluate the rule after scoring

4. Dataset Fields to Use

Use the diabetes hospital readmission dataset provided for the assignment. You do not need every column. Build a focused operational subset.

```
encounter_id
patient_nbr
age
time_in_hospital
number_inpatient
number_emergency
number_outpatient
num_medications
num_lab_procedures
discharge_disposition_id
admission_type_id
readmitted
```

Critical Data Leakage Rule

The `readmitted` field is the historical outcome. Do NOT use it to calculate the risk score. Use it only after scoring to evaluate whether your prioritization identified a population with higher historical <30-day readmission.

5. Prototype Prioritization Rule

For this mini project, use a simple explainable scoring rule. These weights are learning assumptions, not clinical recommendations.

Condition	Points
number_inpatient >= 3	**+3**
number_emergency >= 2	**+2**
number_outpatient >= 3	**+1**
time_in_hospital >= 7	**+1**
num_medications >= 20	**+1**

Risk level mapping: **0-2 = LOW | 3-4 = MEDIUM | 5+ = HIGH**

6. Target Architecture



Layering Rule

Keep SQL in repository.py, business rules in service.py, request/response handling in main.py, and database configuration in database.py.

7. PostgreSQL Design

Table A — patient_encounters

```
encounter_id
patient_nbr
age
time_in_hospital
number_inpatient
number_emergency
number_outpatient
num_medications
num_lab_procedures
discharge_disposition_id
admission_type_id
readmitted
risk_score
risk_level
```

Table B — care_interventions

```
intervention_id
encounter_id
status
assigned_to
notes
created_at
updated_at
```

Suggested intervention statuses:

```
PENDING_REVIEW → ASSIGNED → CONTACTED → FOLLOW_UP_SCHEDULED → CLOSED
```

8. What You Must Build

1. Load the selected healthcare dataset fields into PostgreSQL.
2. Calculate risk_score and risk_level without using the historical readmitted outcome.
3. Create a care-management queue for MEDIUM and HIGH risk encounters.
4. Allow care-management intervention status to be tracked.
5. Expose operational metrics through FastAPI.
6. Compare your prioritization against historical <30-day readmission outcomes.

9. Mandatory FastAPI Endpoints

Method	Endpoint	Purpose
GET	/encounters	List encounters
GET	/encounters/{encounter_id}	Get one encounter
GET	/care-queue	Return MEDIUM + HIGH risk queue
GET	/care-queue/high	Return only HIGH risk queue
PATCH	/interventions/{encounter_id}	Update workflow status / owner / notes
GET	/operations/summary	Operational metrics
GET	/operations/readmission-analysis	Compare prioritization vs historical outcomes
GET	/management/dashboard	Executive summary metrics

10. Example Workflow

Example intervention update:

```
PATCH /interventions/12345

{
  "status": "CONTACTED",
  "assigned_to": "Care Team A",
  "notes": "Follow-up appointment scheduled"
}
```

Example operations summary:

```
{
  "total_encounters": 10000,
  "high_risk": 850,
  "medium_risk": 2100,
  "low_risk": 7050,
  "pending_interventions": 640,
  "completed_interventions": 210
}
```

Example readmission analysis:

```
{
  "high_risk_patients": 850,
  "high_risk_readmitted_under_30_days": 220,
  "high_risk_readmission_rate": 25.88
}
```

Important

The numbers above are response-shape examples only. Your application must calculate metrics from the dataset and database. Do not copy these sample values as results.

11. Acceptance Criteria

- Dataset can be loaded into PostgreSQL without manually inserting individual encounters.
- Risk score is calculated without using `readmitted`.
- MEDIUM and HIGH risk cases appear in the care queue.
- Intervention status can be updated and remains persisted after FastAPI restarts.
- Operational summary is calculated from stored data.
- Readmission analysis compares risk groups against historical `<30` outcomes.
- Routes return useful HTTP errors for missing encounter IDs.
- SQL is parameterized; no user-supplied values are concatenated into SQL strings.
- Code follows main / models / service / repository / database separation.

12. Expected Project Structure

```
healthcare-readmission-mvp/
|
 ├── main.py
 ├── models.py
 ├── service.py
 ├── repository.py
 ├── database.py
 ├── data_loader.py
 ├── schema.sql
 ├── seed_or_setup.sql
 ├── requirements.txt
 ├── .env.example
 ├── README.md
 └── data/
   └── <provided diabetes dataset>
```

13. Submission Deliverables

- GitHub repository or zipped source repository
- README.md with business problem, users, assumptions, architecture, API list and run instructions
- schema.sql
- data_loader.py
- FastAPI source files
- Swagger screenshot showing key endpoints
- PostgreSQL screenshot showing loaded records
- 3-5 minute demo video

14. Evaluation Rubric

Area	Weight	What Good Looks Like
FDE problem understanding	15%	Clear user, decision, unknowns and MVP assumptions
Dataset ingestion / cleanup	10%	Focused columns, repeatable loading, sensible missing-data handling
PostgreSQL design	15%	Usable schema and persistence
Business / risk rules	15%	Explainable, isolated in service layer, no leakage
FastAPI + layering	20%	Clean API, repository separation, useful errors
Care-management workflow	10%	Queue and intervention lifecycle work end-to-end
Operational metrics	10%	Metrics are calculated from data, not hard-coded
README / demo quality	5%	Easy to run and easy to explain

15. Stretch Goals — Optional

- Filter care queue by minimum risk score.
- Add pagination to encounter APIs.
- Add summary by age band or utilization history.
- Add an intervention history table instead of overwriting the latest status.
- Add a CSV export endpoint for the care queue.
- Compare the simple rule against an alternative rule set and explain which is more useful operationally.

16. FDE Reflection — Required in README

7. What business problem did you solve?
8. Who is the primary user?
9. What decision does your application improve?
10. What did the dataset prove?
11. What assumptions did you make?
12. What must the hospital validate before this could become a real product?
13. Would you use rules, traditional ML, LLM, or another approach next — and why?

Final Goal

Your objective is not to prove that you can build CRUD APIs. Your objective is to show that you can convert enterprise healthcare data into an operational decision workflow with measurable outcomes.

LEARNING USE ONLY — NOT FOR CLINICAL DECISION-MAKING